This material was developed in the context of the Nordplus project NPHE-2025/10158, “Co-designing a Computational Thinking and Mathematics Curriculum Framework for Nordic-Baltic Preservice Teacher Education

Activity 2: Exploring Polygons Through Scratch

Rationale: Why are PSTs expected to learn this? Why are the outcomes of this lesson important? Why are these outcomes essential for future learning?

Explore mathematical properties of regular polygons using Scratch. PSTs will teach these properties, which is part of the national curriculum. Future teachers need more than procedural knowledge of Scratch or isolated facts about polygons. They need pedagogical content knowledge that enables them to connect geometric relationships, algorithmic representations, pupils' reasoning, and purposeful use of technology. Regular polygons provide a productive context because equal side lengths, repeated turns, interior and exterior angles, and total rotation can be represented directly in code.

Scratch makes mathematical structure visible and testable. A repeating loop conveys the regularity of the geometrical shape; an incorrect turn provides immediate visual evidence; and changing the number of sides invites PSTs to move from individual examples to the general rule: turning angle = 360°/n. This supports prediction, explanation, testing, debugging, and generalisation rather than trial-and-error coding alone.

Aims and objective:

Towards which goal are PSTs expected to learn?

By the end of the activity, PSTs will be able to:

- Explain the geometric properties of regular polygons and distinguish an interior angle from the sprite’s exterior/turning angle;
- Derive and justify the rule turning angle = 360°/n and relate it to a complete rotation of 360°;
- Design, test, debug, and improve Scratch programs that use sequence, repetition, operators;
- Identify and explain the Computational Thinking practices involved: decomposition, algorithmic thinking, pattern recognition, testing and debugging, abstraction, and generalisation;

Time:

How long is the lesson? How do you divide the time?

Overall activity: 3 × 45 minutes (135 minutes)

- Launch and diagnostic discussion — 15 min
- Step 1: square — 20 min
- Step 2: equilateral triangle — 20 min
- Step 3: regular hexagon — 20 min
- Step 4: any regular polygon — 30 min
- Closure, synthesis, and exit ticket — 30 min

Optional fourth session (45 min): pedagogical analysis, lesson redesign, or microteaching.

○ Anticipate students' likely misconceptions and plan suitable questions, scaffolds, differentiation, and formative assessment; ○ Evaluate what programming adds to the learning of geometry and when an unplugged or paper-based representation may be useful.	
Content/Topic: **Computational thinking:** decomposition, algorithmic thinking, pattern recognition, testing and debugging, abstraction, and generalisation. Integrated with: **Mathematics:** regular polygons; equal sides and angles; interior and exterior/turning angles; complete rotation; generalisation using n sides. **Programming as a practice of Computational Thinking:** motion and pen blocks; sequence; repeat loops; and operators. **Progression:** ***square → equilateral triangle → regular hexagon → a parameterised program for any regular polygon*****.**	**Assessment:** Assessment should focus not only on whether PSTs produce the required shape, but also on the reasoning demonstrated through their code, explanations, testing, and revisions. The teacher educator gathers evidence by observing each pair's planning, dialogue, code, figures, testing, and revisions. Short questions such as "Why does this angle work?", "What did you change and why?", and "How do you know that the program is correct?" can reveal whether PSTs understand the mathematical relationships or have reached the answer through trial and error. Evidence from pair work should be complemented by the whole-class discussion. The teacher educator can note whether PSTs can compare different solutions, explain the relationship between interior and turning angles, identify the computational thinking processes involved, and discuss how pupils' misconceptions could be addressed. Some misconception examples are provided. Example 1: A pupil wants to draw a regular octagon and uses "repeat 8" and "turn 135°." Identify what is correct and what needs to be changed. Give the correct turning angle, justify it mathematically, and write one question that could help the pupil reconsider the use of 135°.

	Example 2: The program draws a correct square once but fails when run again because the sprite's position, direction, or previous drawing is not reset. Example 3: PSTs use a 60° turn because an equilateral triangle has interior angles of 60° Example 4: PSTs use 120°, the interior angle of a regular hexagon, as the sprite's turning angle. Example 5: PSTs assume that any closed six-sided figure is a regular hexagon. Example 6: PSTs change the repeat count to six but retain the triangle's 120° turn. The program consequently traces a triangle twice. Example 7: PSTs use the number entered by the user directly as the turning angle. For example, they turn 8° for an octagon. Example 8: PSTs create separate scripts for a triangle, square, and hexagon instead of constructing one general algorithm. To obtain evidence of individual learning, each PST completes a short exit card. The exit cards can be used to identify concepts that require further clarification and to inform the planning of follow-up activities. The following check items could also be included in the Exit Card:

	“For each statement, select Not yet / With support / Independently and provide brief evidence from your program or explanation. ● I can explain why the repeat count equals the number of sides. ● I can derive the turning angle rather than determine it through trial and error. ● I can distinguish the interior angle from the sprite’s turning angle. ● I can use an incorrect output to identify what should be revised. ● I can generalise the program for any regular polygon.”

Learning activities:
How are PSTs expected to learn this? What specific activities, investigations, problems, questions, or tasks will pupils be working on during the lesson?

Learning follows a repeated cycle: predict → plan → code → test → explain → compare → generalise → reflect on teaching. PSTs work as mathematical learners first and then examine the same experience from a teacher's perspective.

Lesson launch/Time: Exactly how will you use the first minutes of the lesson? How will you engage the STs?
15 minutes

Activate prior knowledge without giving the formula. Display or physically trace a square and ask one PST to act as the Scratch sprite. PSTs predict the instructions required and discuss what the sprite must do at each vertex.

- What properties make a polygon regular?
- Which quantities belong to the shape, and which describe the sprite's movement?
- If the sprite returns to its starting direction, how far has it turned altogether?
- What would count as convincing evidence that a program is correct?

Main part/Time: Step 1: 20 min, Step 2: 20 min, Step 3: 20 min and Step 4: 30 min

The lesson is prepared by using Scratch. Alternative learning environment: **XLogoOnline:** https://xlogo.inf.ethz.ch/release/v.7.7.6/#/

Step 1: Drawing a Square

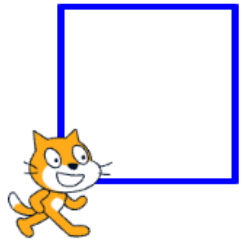
- Before you open Scratch: You will be using Scratch to draw a square. Discuss with your partner

PSTs prior knowledge/Prerequisites:
What prior knowledge do PSTs need?

- Basic use of Scratch: making an account (it is recommended, it can be used without an account also), selecting a sprite, starting a script, motion blocks, control blocks, and adding the Pen extension.
- Meaning of a regular polygon and the properties of squares and equilateral triangles.
- Basic angle language and awareness that one complete turn is 360°.
- Prior knowledge about CT (processes) and links to programming.

- How can you accomplish this?
- What kind of blocks would you need?
- Can you break it down into smaller parts?
- Try to write an outline of the program on paper. Justify your solution.

- Now you can open Scratch: Add the pen-blocks to your project and use this to make the cat draw a square.
  - How do you know your solution is correct?
  - Can you improve your code in any way? Make it more efficient?
  - Look to your neighbour - do you have the same solution or are there some differences?
  - Did you notice that the sprite's starting position is important for further steps?



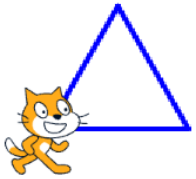
After Step 1, the teacher educator conducts a brief whole-class discussion:

- How did you determine that the program correctly draws a square?

Step 2: Drawing an equilateral triangle

- Look at your code. How can you change your code so that the cat instead draws an equilateral triangle? Discuss with your learning partner.
  - What happens if you use a 60-degree turn? Try it and explain the resulting path.

- What part of your code did you have to change? What could you keep? Justify.
- Is the angle through which the cat turns the same as the interior angle of the triangle? Explain.
- Can you describe a general rule connecting the number of sides of a regular polygon and the angle through which the cat needs to turn?
- Why 120°?, Why does the total turning equal 360°?, and what parts of the program represent mathematical properties of the triangle? These questions connect geometry directly with algorithmic thinking.



After Step 2, the teacher educator conducts a brief whole-class discussion:

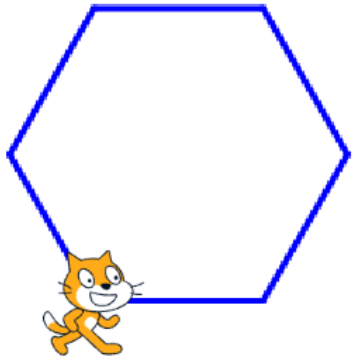
- Why does the sprite turn 120° rather than 60° when drawing an equilateral triangle?
- How many times should the movement-and-turn sequence be repeated?
- A regular hexagon has interior angles of 120 degrees. Does the cat turn 120 degrees? Why or why not?
- What is the relationship between the interior angle and the turning angle?

Step 3: Drawing a Regular Hexagon

Now we will try to obtain a hexagon.

- Look at your code in the previous step. How can you change your code so that the cat instead draws a Hexagon? Discuss with your learning partner.

- What part of your code did you have to change? What could you keep? Justify.



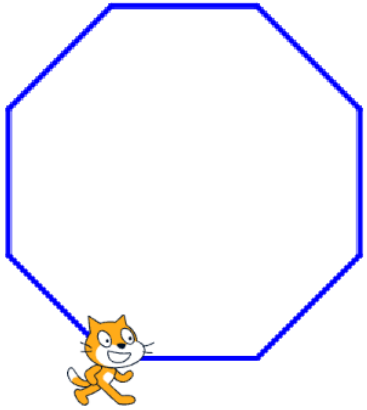
After Step 3, the teacher educator conducts a brief whole-class discussion:

- Why must the total turning be 360 degrees?
- How are the number of sides, the repeat count, and the turning angle related in the hexagon program?
- What patterns across the square, triangle, and hexagon helped you move towards a general rule?

Step 4: Drawing any Regular Polygon

- One final change. Now the user of the program should be able to input the number of angles they want, and the cat will draw that particular regular polygon.
  - What part of your code did you have to change? What could you keep? Justify.
  - Switch computers with your neighbour. Is your solution the same? What are the differences? Can you improve your program based on your neighbours work? Can your work help your neighbour?

- Can you find a rule for calculating the turning angle of any regular polygon from its number of sides?



(An exemplary Octagon)

After Step 4, the teacher educator conducts a brief whole-class discussion:

- Why does the program work for any regular polygon entered by the user? When not?

Notes for discussion at the end of each Step:
Depending on the PSTs' prior knowledge and the available time, the teacher educator may then discuss the following questions or revisit them in a follow-up lesson:

- Which computational thinking processes were involved in this step, and where were they evident in the activity?
- How did these processes support the development or improvement of the program?

Step 5: Closure

30 min for discussion and to conclude. Exemplary Questions:

• Focusing on CT processes: *Which CT processes were involved—decomposition, algorithmic thinking, pattern recognition, debugging, generalisation*? • The role of programming: *What did programming make visible that might be less visible in a traditional geometry activity*? • Focusing on some possible misconceptions: For example, *why might pupils use 120 degrees instead of 60 degrees for the hexagon*? *How could a teacher respond without simply giving the answer*?	

Grouping and Location:

With whom are PSTs expected to learn? Is grouping going to vary during the lesson? Where are PSTs learning?

The lesson is designed for a face-to-face computer room or a bring-your-own-device setting. PSTs work mainly in pairs, ideally with one device per pair, because paired prediction and explanation make reasoning audible and reduce passive participation. Roles such as driver and navigator should be exchanged at each step.

Pairs periodically compare solutions with a neighbouring pair, while whole-class discussion is used at the launch, after critical misconceptions emerge, and during closure. The teacher educator may use purposeful pairing to balance Scratch experience, but should monitor that the more experienced partner does not take over the keyboard or the reasoning.

Teacher educator's facilitation:

How is the teacher educator facilitating PSTs' learning?

The teacher educator facilitates PSTs' learning by combining guided inquiry, mathematical discussion, and reflection. A strong facilitation sequence would be: explore → predict → test → explain → compare → generalise → reflect on teaching. This keeps PSTs active as mathematical learners while also developing their pedagogical content knowledge. The teacher educator also guides PSTs to think about assessment, differentiation, and lesson adaptation for different age groups and levels of prior knowledge.

The teacher educator combines guidance, mathematical discussion, and explicit pedagogical reflection. The central facilitation sequence is: explore → predict → test → explain → compare → generalise → reflect on teaching.

- Launch each step with a prediction or planning prompt before PSTs touch the code.
- Circulates and listens for evidence of reasoning, asking "Why?", "How do you know?", and "What does this block represent mathematically?" rather than immediately correcting errors.
- Makes the distinction between interior and turning angles explicit only after PSTs have had an opportunity to reason about it.

- Supports access through diagrams, physical turning, sentence starters, partially completed code, or extension prompts, while keeping the same core learning goal.
- Shift the discussion from “How did we solve it?” to “How would we teach and assess it?”, thereby developing pedagogical content knowledge.

Material and Resources:

What materials and/or resources are essential/beneficial for PSTs (and the teacher educator) to successfully accomplish the learning goals? And why?

Essential materials:

- One internet-connected computer or tablet per pair, with Scratch available in a browser or as the offline desktop application.
- A prepared starter project or common starting instructions, plus access to Motion, Control, Operators, Sensing, Variables, and the Pen extension.
- An investigation sheet with space to record predictions, values of n, turning angles, observed results, explanations, and revisions.
- A shared display for comparing programs and figures; paper polygons, rulers, protractors, and optional floor space for physically modelling turns.
- A short observation checklist and exit ticket aligned with the assessment criteria.

Recommended online resources:

Scratch Editor https://scratch.mit.edu/projects/editor/ — the environment in which PSTs construct and test the programs
Scratch ideas https://scratch.mit.edu/ideas — short tutorials and starter activities for beginners
Scratch for Educators: https://www.scratchfoundation.org/learn/learning-library
XLogoOnline: https://xlogo.inf.ethz.ch/release/v.7.7.6/#/
Computational Thinking with Scratch: https://scratched.gse.harvard.edu/ct/index.html
Google Computer Science Education (Computational Thinking): https://edu.google.co.in/cs/learn/educator.html
CSUnplugged activities: https://www.csunplugged.org/en/
Many CT tasks can be found here: https://www.fsf.vu.lt/ct-math-able#creating-a-set-of-interactive-tasks — examples connecting computational thinking and mathematics

Follow up activities

What questions/activities are appropriate for the PSTs to do after the lesson?/ What activity will reinforce the learning goals?

Suitable follow-up activities could include:

- Modify the activity assignment/further teaching:

    - How could the task be adapted for younger pupils, more advanced pupils, or pupils who need additional support?
- Reflect on technology use assignment/discussion or further teaching:
    - What does programming add to the learning of geometry here? Could the same mathematical goal be achieved without programming, and what would be different?
    - Microteaching: in groups of three, one PST teaches, one acts as a pupil with a planned misconception, and one observes the questioning and feedback. Rotate roles and discuss the evidence of learning.